ML Problem Solving

Stochastic Gradient Descent

Part 1: Mathematical Foundations & Derivations

Context & Problem Setup

We consider an image classification task where each observation is represented by a 4-pixel image feature vector:

$$\mathbf{x} = [x_1, x_2, x_3, x_4]^T \in \mathbb{R}^4$$

The target variable is a multiclass label $y \in \{0, 1, 2\}$, representing $C = 3$ mutually exclusive classes.

The model is a single-layer neural network parameterized by a weight matrix $\mathbf{W} \in \mathbb{R}^{4\times 3}$ and a bias vector $\mathbf{b} \in \mathbb{R}^{1\times 3}$:

$$\mathbf{W} = \begin{bmatrix} w_{10} & w_{11} & w_{12} \\ w_{20} & w_{21} & w_{22} \\ w_{30} & w_{31} & w_{32} \\ w_{40} & w_{41} & w_{42} \end{bmatrix}, \quad \mathbf{b} = [b_0, b_1, b_2]$$

For a row sample vector $\mathbf{x}^T = [x_1, x_2, x_3, x_4]$, the linear logit vector $\mathbf{z} \in \mathbb{R}^{1\times 3}$ is given by:

$$z_j = b_j + \mathbf{x}^T \mathbf{w}_{:,j} = b_j + \sum_{i=1}^{4} x_i w_{ij}, \quad \forall j \in \{0, 1, 2\}$$

The predicted class posterior probabilities $\mathbf{p} = [p_0, p_1, p_2]$ are computed via the Softmax function:

$$p_j = \sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=0}^{C-1} e^{z_k}}$$

Question 1.1: Target Representation via One-Hot Encoding

1. Define the formal transformation mapping scalar label $y \in \{0, 1, \ldots, C-1\}$ to its one-hot vector representation $\mathbf{y} \in \{0, 1\}^C$.

2. Write down the explicit one-hot encoded vector targets $\mathbf{y}^{(k)}$ for each sample in the provided training set D_{train}:
   - Sample 1: $[255, 128, 128, 0]$, $y = 0$
   - Sample 2: $[55, 128, 128, 128]$, $y = 1$
   - Sample 3: $[192, 128, 128, 0]$, $y = 0$
   - Sample 4: $[100, 128, 128, 100]$, $y = 1$
   - Sample 5: $[30, 64, 128, 30]$, $y = 2$
   - Sample 6: $[20, 64, 128, 0]$, $y = 2$

3. Prove that the multiclass Categorical Cross-Entropy loss for a single sample, defined as:

$$L(\mathbf{y}, \mathbf{p}) = -\sum_{k=0}^{C-1} y_k \ln(p_k)$$

   collapses strictly to $L = -\ln(p_c)$ where c is the true class index.

Question 1.2: Rigorous Gradient Derivations (optional)

Using the chain rule of multivariate calculus: 1. Compute the Jacobian $\frac{\partial p_k}{\partial z_j}$ of the Softmax function, distinguishing between the cases $k = j$ and $k \neq j$. 2. Derive the derivative of the single-sample cross-entropy loss with respect to logit z_j:

$$\frac{\partial L}{\partial z_j} = p_j - y_j$$

3. Derive the analytical expressions for the parameter gradients:

$$\frac{\partial L}{\partial w_{ij}} \quad \text{and} \quad \frac{\partial L}{\partial b_j}$$

4. Show how the parameter update rules with learning rate α yield:

$$\Delta w_{ij} = -\alpha \frac{\partial L}{\partial w_{ij}} = \alpha x_i (y_j - p_j)$$

$$\Delta b_j = -\alpha \frac{\partial L}{\partial b_j} = \alpha (y_j - p_j)$$

Verify that this matches the piecewise notation provided in the original workshop problem statement:

$$y_j - p_j = \begin{cases} 1 - p_j, & \text{if } j = \text{ground-truth class } a \\ -p_j, & \text{otherwise} \end{cases}$$

Part 2: Manual Tracing — Pure Stochastic Gradient Descent (SGD)

Training Protocol

- **Initial Weights and Biases:** All parameters initialized to zero:

$$\mathbf{W}^{(0)} = \mathbf{0}_{4\times 3}, \quad \mathbf{b}^{(0)} = \mathbf{0}_{1\times 3}$$

- **Learning Rate:** $\alpha = 0.1$
- **Update Cadence:** Pure SGD updates weights and biases immediately after processing **each individual training sample**.
- **Sample Order:** Traverse the dataset sequentially from Sample 1 through Sample 6.

Dataset Specifications

$$D_{\text{train}} = \left(\begin{array}{cccc|c} 255 & 128 & 128 & 0 & 0 \\ 55 & 128 & 128 & 128 & 1 \\ 192 & 128 & 128 & 0 & 0 \\ 100 & 128 & 128 & 100 & 1 \\ 30 & 64 & 128 & 30 & 2 \\ 20 & 64 & 128 & 0 & 2 \end{array}\right)$$

$$D_{\text{val}} = \left(\begin{array}{cccc|c} 155 & 64 & 64 & 32 & 0 \\ 0 & 128 & 128 & 192 & 1 \\ 20 & 40 & 80 & 20 & 2 \end{array}\right)$$

Question 2.1: Epoch 1 Trace

Calculate the step-by-step parameter trajectory across all 6 samples for Epoch 1. For every step $t = 1, 2, \ldots, 6$: 1. Write down current logits $\mathbf{z} = [z_0, z_1, z_2]$. 2. Compute predicted probabilities $\mathbf{p} = [p_0, p_1, p_2]$. 3. Compute the error vector $\mathbf{e} = \mathbf{y} - \mathbf{p}$. 4. Compute the updates $\Delta\mathbf{b}$ and $\Delta\mathbf{W}$. 5. Provide the updated parameter matrices $\mathbf{W}^{(t)}$ and $\mathbf{b}^{(t)}$.

Keep track of your intermediate values in a structured tabular format.

Question 2.2: Epoch 2 Trace

Continue the sequential SGD updates for a complete second pass (Epoch 2) using the exact parameter values obtained at the end of Epoch 1. 1. Show the step-by-step update calculations for each sample. 2. Report the final state of $\mathbf{W}$ and $\mathbf{b}$ after Epoch 2 completes.

Question 2.3: Training Accuracy & Empirical Loss

Using the final weights and biases obtained after Epoch 2: 1. Compute the forward pass (logits and argmax prediction $\hat{y} = \arg\max_j z_j$) for all 6 training samples. 2. Determine the classification accuracy on D_{train}. 3. Compute the average empirical cross-entropy loss over the training set:

$$L_{\text{train}} = \frac{1}{|D_{\text{train}}|} \sum_{n=1}^{6} -\ln(p_{y^{(n)}}^{(n)})$$

Question 2.4: Validation Evaluation

Evaluate the model obtained after Epoch 2 on the validation set D_{val}: 1. Compute the logit vectors $\mathbf{z}$ and Softmax probabilities $\mathbf{p}$ for each validation sample. 2. Provide predicted class labels $\hat{y}$ versus ground-truth labels y. 3. Compute the Validation Accuracy. 4. Calculate the average validation cross-entropy loss.

Part 3: Manual Tracing — Mini-Batch Gradient Descent

Now, reset all model parameters back to zero:

$$\mathbf{W}^{(0)} = \mathbf{0}_{4\times 3}, \quad \mathbf{b}^{(0)} = \mathbf{0}_{1\times 3}$$

We now execute **Mini-Batch Gradient Descent** with a batch size of $B = 3$ and learning rate $\alpha = 0.1$. * **Batch 1 ($\mathcal{B}_1$):** Samples 1, 2, and 3. * **Batch 2 ($\mathcal{B}_2$):** Samples 4, 5, and 6.

Recall that the batch-averaged gradient is defined as:

$$\nabla_{\mathbf{W}} L_{\mathcal{B}} = \frac{1}{B} \sum_{k\in\mathcal{B}} \nabla_{\mathbf{W}} L^{(k)}, \quad \nabla_{\mathbf{b}} L_{\mathcal{B}} = \frac{1}{B} \sum_{k\in\mathcal{B}} \nabla_{\mathbf{b}} L^{(k)}$$

and parameter updates are applied once per mini-batch:

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha \nabla_{\mathbf{W}} L_{\mathcal{B}}, \quad \mathbf{b} \leftarrow \mathbf{b} - \alpha \nabla_{\mathbf{b}} L_{\mathcal{B}}$$

Question 3.1: Mini-Batch Gradient Descent Execution (1 Epoch)

1. **Batch 1 Step:**
   - Evaluate all 3 samples in $\mathcal{B}_1$ under the initial parameter state $\mathbf{W}^{(0)}, \mathbf{b}^{(0)}$.
   - Calculate individual error vectors $\mathbf{e}^{(1)}, \mathbf{e}^{(2)}, \mathbf{e}^{(3)}$.
   - Compute the batch-averaged parameter gradients $\nabla_{\mathbf{W}} L_{\mathcal{B}_1}$ and $\nabla_{\mathbf{b}} L_{\mathcal{B}_1}$.
   - Apply the update to compute $\mathbf{W}^{(1)}$ and $\mathbf{b}^{(1)}$.
2. **Batch 2 Step:**
   - Evaluate samples 4, 5, and 6 in $\mathcal{B}_2$ using the updated parameters $\mathbf{W}^{(1)}, \mathbf{b}^{(1)}$.
   - Compute the batch-averaged parameter gradients $\nabla_{\mathbf{W}} L_{\mathcal{B}_2}$ and $\nabla_{\mathbf{b}} L_{\mathcal{B}_2}$.
   - Apply the update to obtain the final parameters after 1 epoch: $\mathbf{W}^{(2)}$ and $\mathbf{b}^{(2)}$.

Question 3.2: Mini-Batch Evaluation

1. Compute the training accuracy and validation accuracy using the model weights trained via Mini-Batch Gradient Descent.
2. Compare the parameter magnitudes between Pure SGD (after 1 epoch) and Mini-Batch GD (after 1 epoch). Why do their magnitudes and trajectories diverge?

Part 4: PyTorch Implementation & Autograd Mechanics

In this section, your team will translate the manual algorithmic steps into clean, modular, modern PyTorch code.

Question 4.1: Dataset & Pipeline Construction

Implement a custom PyTorch `Dataset` and `DataLoader` for D_{train} and D_{val}. 1. Ensure your inputs are typed as `torch.float32` and targets as `torch.long`. 2. Configure data loaders to support both sequential SGD ($B = 1$, `shuffle=False`) and mini-batch loading ($B = 3$).

Question 4.2: Pure SGD Implementation from Scratch (No `nn.Module`, No `torch.optim`)

Implement the forward pass, cross-entropy calculation, and gradient backpropagation using raw tensors and `autograd`: 1. Initialize parameters $\mathbf{W}$ and $\mathbf{b}$ with `requires_grad=True`. 2. Loop over the training samples one by one for 2 epochs. 3. Verify that after step 1, `W.grad` and `b.grad` match your analytical derivation from Question 2.1 to within 10^{-5}. 4. Update weights inside a `torch.no_grad()` block and zero out gradients manually via `.grad.zero_()`.

Question 4.3: Refactored PyTorch Standard Model

Refactor the model using standard idiomatic PyTorch: 1. Define a class `LinearClassifier(nn.Module)`. 2. Train the model using `nn.CrossEntropyLoss()` and `torch.optim.SGD`. 3. Plot the training loss trajectory over 20 epochs for learning rates $\alpha \in \{0.0001, 0.001, 0.01, 0.1\}$.